INTRODUCTION

As students every one of us we often struggle to stay organized with all our schoolwork, assignments, and exams. It's easy to forget deadlines or lose track of what needs to be done, especially when everything starts piling up.

This software "To do list" aims to make the process easier by giving students one simple place to keep track of their tasks and grades. The app helps users add their assignments, sort them by priority and add it to the calendar, and even track their grades to see how they're doing overall. This makes it much easier to plan ahead and stay on top of everything without feeling overwhelmed.

This app not only helps students manage their time better but also motivates them to stay focused and organized. By keeping all their work in one spot and showing their progress clearly, our "To-do list" can help reduce stress and make studying feel more manageable.

Overall, goal was to create a helpful and easy-to-use tool that supports students in improving their productivity and keeping better control over their academic performance.

REQUIREMENTS

Software Requirements:

- Windows 7 or higher or macOS or Linux
- Python 3.10 or higher
- Flask module
- Sqlite3 module
- MYSQL 8.4 or higher
- Code Editors such as Visual Studio Code or PyCharm is preferred
- Web browser such as Edge or Chrome

Hardware Requirements:

- Keyboard
- Mouse
- Monitor
- Processor- Dual core or better
- Graphic Card- Not required
- Storage
    - Hard disk-10MB of free space
    - RAM- 2GB

SOURCE CODE

```
from flask import Flask, render_template, request, redirect, url_for,
session, jsonify
import sqlite3

app = Flask(____name____)
app.secret_key  =  'your_secret_key'


def init_db():
    conn = sqlite3.connect('database.db') c =
    conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS students (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT NOT NULL UNIQUE, password
            TEXT NOT NULL, theme_preference TEXT
            DEFAULT 'dark'
        )
    ''')
    c.execute('''
        CREATE TABLE IF NOT EXISTS classes (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            student_id INTEGER,
```

```
        name TEXT, day
        TEXT,
        start_time TEXT,
        end_time TEXT,
        status TEXT DEFAULT 'todo',
        FOREIGN KEY (student_id) REFERENCES students(id)
    )
'''')

# New table for storing grades
c.execute('''
    CREATE TABLE IF NOT EXISTS grades (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        student_id INTEGER,
        exam_name TEXT,
        course_name TEXT,
        score REAL, -- Use REAL for floating point scores total_score
        REAL, -- Use REAL for floating point total scores FOREIGN KEY
        (student_id) REFERENCES students(id)
    )
''')

# Add columns if they don't exist (basic migration attempt) try:
    c.execute("SELECT theme_preference FROM students LIMIT

1") except sqlite3.OperationalError:
    c.execute("ALTER TABLE students ADD COLUMN
theme_preference TEXT DEFAULT 'dark'") try:
```

```
        c.execute("SELECT status FROM classes LIMIT 1")
    except sqlite3.OperationalError:
        c.execute("ALTER TABLE classes ADD COLUMN status TEXT
DEFAULT 'todo'")

    # Check if grades table exists (more robust check than just  ALTER)
    c.execute("SELECT name FROM sqlite_master WHERE  type='table'
AND name='grades'")
    if c.fetchone() is None:
        # Table doesn't exist, init_db already created it, but this check # is
        here to show how you might check for table existence if
needed elsewhere.
        pass # Table was created by the CREATE TABLE IF NOT EXISTS
above

    conn.commit()
    conn.close()

init_db()

@app.route('/') def
home():
    if 'user_id' in session:
        return redirect(url_for('dashboard')) return
    redirect(url_for('login'))
@app.route('/register', methods=['GET', 'POST']) def
register():
    if request.method == 'POST':
```

```
        username = request.form['username']
        password = request.form['password']

        if not username or not password:
            return render_template('register.html', error="Username  and
password are required.")

        with sqlite3.connect('database.db') as conn: c =
            conn.cursor()
            c.execute("SELECT id FROM students WHERE username=?",
(username,))
            if c.fetchone():
                return render_template('register.html', error="Username
already exists. Please choose a different one.")

            c.execute("INSERT INTO students (username, password)  VALUES
(?, ?)", (username, password))
            conn.commit()
        return redirect(url_for('login'))  return
    render_template('register.html')


@app.route('/login', methods=['GET', 'POST']) def
login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']

        with sqlite3.connect('database.db') as conn:
            conn.row_factory  =  sqlite3.Row
            c = conn.cursor()
```

```
        c.execute("SELECT id, theme_preference FROM students
WHERE username=? AND password=?", (username, password))
        user = c.fetchone() if
        user:
            session['user_id'] = user['id'] session['theme_preference'] =
            user['theme_preference'] return
            redirect(url_for('dashboard'))
        else:
             return render_template('login.html', error="Invalid username
or password.")

    return render_template('login.html')


@app.route('/dashboard', methods=['GET', 'POST']) def
dashboard():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user_id = session['user_id']
    theme_preference = session.get('theme_preference', 'dark')


    if request.method == 'POST': name
        = request.form.get('name') day =
        request.form.get('day')
        start_time = request.form.get('start_time') end_time
        = request.form.get('end_time')

        if name and day and start_time and end_time: with
            sqlite3.connect('database.db') as conn:
                c = conn.cursor()
```

```
            c.execute("INSERT INTO classes (student_id, name, day,
start_time, end_time, status) VALUES (?, ?, ?, ?, ?, ?)",
                      (user_id, name, day, start_time, end_time, 'todo'))
            conn.commit()
        return redirect(url_for('dashboard')) else:
        print("Missing task fields")
        pass


    with sqlite3.connect('database.db') as conn:
        conn.row_factory = sqlite3.Row
        c = conn.cursor()
        c.execute("SELECT * FROM classes WHERE student_id=?",
(user_id,))
        tasks = c.fetchall()

    return render_template('dashboard.html', tasks=tasks,
theme_preference=theme_preference)


@app.route('/update_task_status', methods=['POST']) def
update_task_status():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Not logged in'}), 401

    user_id = session['user_id'] data =
    request.json

    task_id = data.get('task_id')
    new_status = data.get('status')
```

```
    if not task_id or new_status not in ['todo', 'completed']:
        return jsonify({'success': False, 'message': 'Invalid request data'}),
400

    try:
        with sqlite3.connect('database.db') as conn: c =
            conn.cursor()
            c.execute("UPDATE classes SET status = ? WHERE id = ?
AND student_id = ?",
                      (new_status, task_id, user_id))
            conn.commit()
            if c.rowcount == 0:
                return jsonify({'success': False, 'message': 'Task not found or
does not belong to user'}), 404
        return jsonify({'success': True, 'message': 'Task status updated'})

    except Exception as e:
        print(f"Error updating task status: {e}")
        return jsonify({'success': False, 'message': 'Internal server error'}),
500


@app.route('/save_theme_preference', methods=['POST']) def
save_theme_preference():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Not logged in'}), 401

    user_id = session['user_id'] data =
    request.json

    theme = data.get('theme')
```

```
    if theme not in ['dark', 'light']:
        return jsonify({'success': False, 'message': 'Invalid theme value'}),
400

    try:
        with sqlite3.connect('database.db') as conn: c =
            conn.cursor()
            c.execute("UPDATE students SET theme_preference = ?
WHERE id = ?",
                      (theme, user_id))
            conn.commit()

            session['theme_preference'] = theme

        return jsonify({'success': True, 'message': 'Theme preference saved'})

    except Exception as e:
        print(f"Error saving theme preference: {e}")
        return jsonify({'success': False, 'message': 'Internal server error'}),
500


# New route to handle adding a grade
@app.route('/add_grade', methods=['POST']) def
add_grade():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Not logged in'}), 401

    user_id = session['user_id'] data =
    request.json
```

```
    exam_name = data.get('examName')
    course_name = data.get('courseName')
    score = data.get('score')
    total_score = data.get('totalScore')

    # Basic validation
    if not exam_name or not course_name or score is None or
total_score is None or score < 0 or total_score <= 0 or score >
total_score:
        return jsonify({'success': False, 'message': 'Invalid grade data'}),
400

    try:
        with sqlite3.connect('database.db') as conn: c =
            conn.cursor()
            c.execute("INSERT INTO grades (student_id, exam_name,
course_name, score, total_score) VALUES (?, ?, ?, ?, ?)",
                      (user_id, exam_name, course_name, score, total_score))
            conn.commit()
            # Get the ID of the newly inserted row grade_id =
            c.lastrowid
            return jsonify({'success': True, 'message': 'Grade added
successfully', 'grade': {'id': grade_id, 'examName': exam_name,
'courseName': course_name, 'score': score, 'totalScore': total_score}})

    except Exception as e: print(f"Error
        adding grade: {e}")
        return jsonify({'success': False, 'message': 'Internal server error'}),
500
```

```
# New route to handle getting grades
@app.route('/get_grades', methods=['GET']) def
get_grades():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Not logged in'}), 401

    user_id = session['user_id']

    try:
        with sqlite3.connect('database.db') as conn: conn.row_factory =
            sqlite3.Row # To access columns by
name
            c = conn.cursor()
            c.execute("SELECT * FROM grades WHERE student_id=?",
(user_id,))
            grades = c.fetchall()
            # Convert rows to a list of dictionaries for JSON response
            grades_list = [dict(row) for row in grades]
        return jsonify({'success': True, 'grades': grades_list})

    except Exception as e:  print(f"Error
        getting grades: {e}")
        return jsonify({'success': False, 'message': 'Internal server error'}),
500

# New route to handle deleting a grade
@app.route('/delete_grade', methods=['POST']) def
delete_grade():
    if 'user_id' not in session:
        return jsonify({'success': False, 'message': 'Not logged in'}), 401

    user_id = session['user_id']
```

```
    data = request.json

    grade_id = data.get('grade_id') if

    grade_id is None:
        return jsonify({'success': False, 'message': 'Invalid request data'}),
400

    try:
        with sqlite3.connect('database.db') as conn: c =
            conn.cursor()
            # Delete the grade, ensuring it belongs to the logged-in user
            c.execute("DELETE FROM grades WHERE id = ? AND
student_id = ?",
                      (grade_id, user_id))
            conn.commit()
            if c.rowcount == 0: # Check if any row was deleted
                return jsonify({'success': False, 'message': 'Grade not found or
does not belong to user'}), 404
        return jsonify({'success': True, 'message': 'Grade deleted
successfully'})

    except Exception as e: print(f"Error
        deleting grade: {e}")
        return jsonify({'success': False, 'message': 'Internal server error'}),
500


@app.route('/settings')
def settings():
    if 'user_id' not in session:
        return redirect(url_for('login'))
```

```
    theme_preference = session.get('theme_preference', 'dark')

    return render_template('settings.html',
theme_preference=theme_preference)


@app.route('/focus') def
focus():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    theme_preference = session.get('theme_preference', 'dark') return

    render_template('focus.html',
theme_preference=theme_preference)


@app.route('/stats')
def stats(): # Corrected function name if
    'user_id' not in session:
        return redirect(url_for('login'))

    theme_preference = session.get('theme_preference', 'dark')  return

    render_template('stats.html',
theme_preference=theme_preference) # Corrected template name


@app.route('/logout') def
logout():
    session.clear()
```

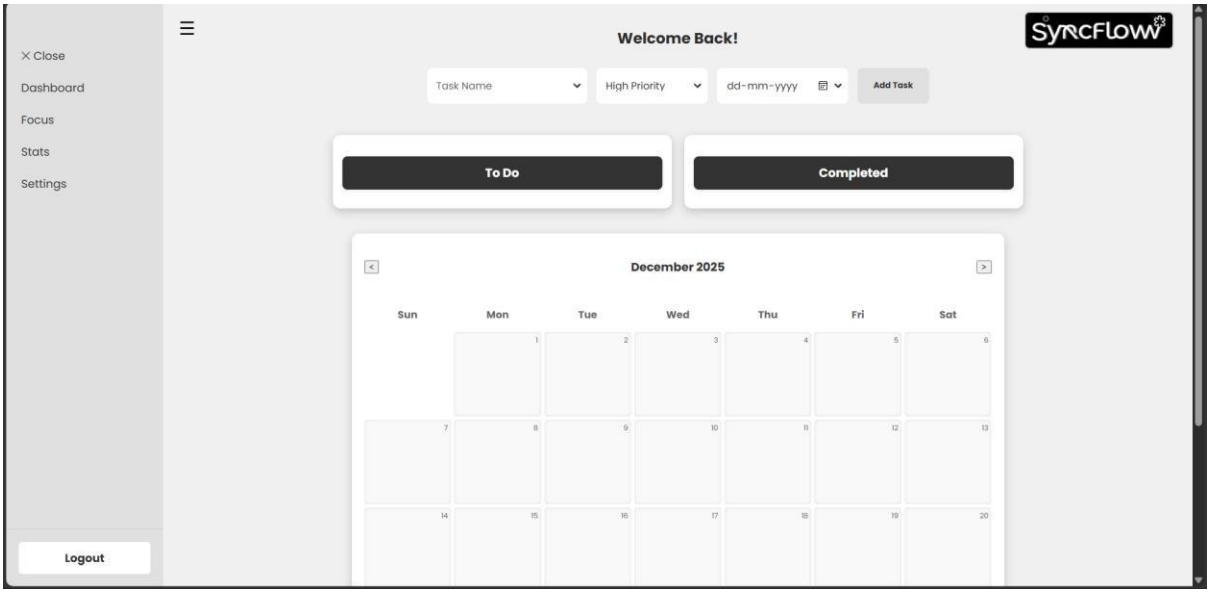
```
    return redirect(url_for('login'))


if __name__ == '__main__':
    app.run(debug=True, port = "5070")
```

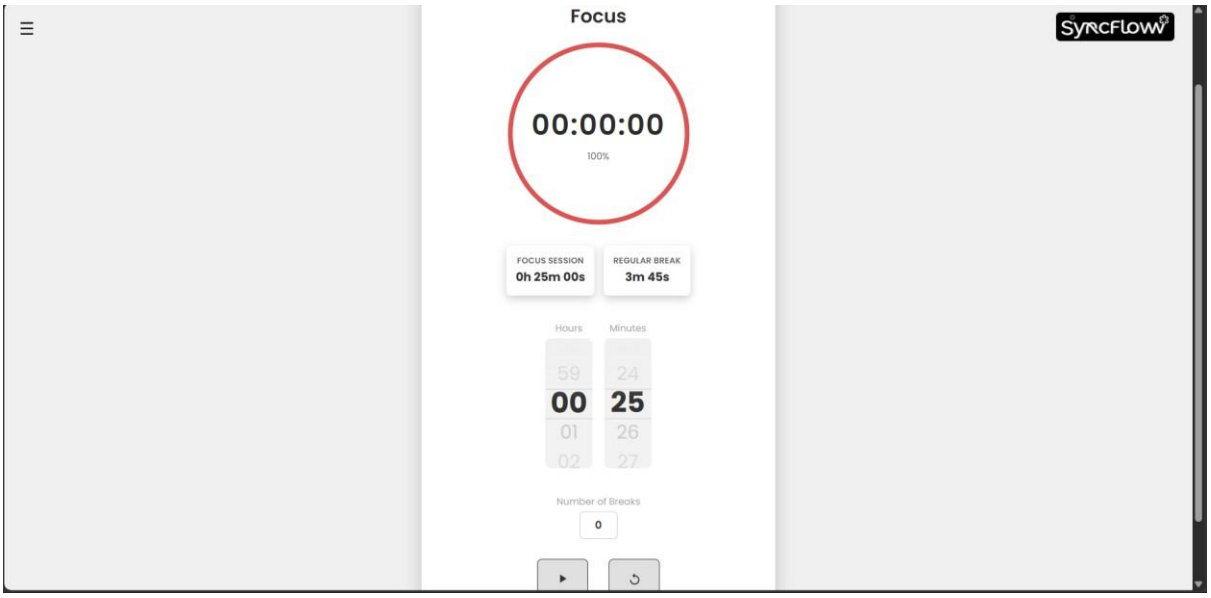
OUTPUT SCREENSHOTS

Registration Page

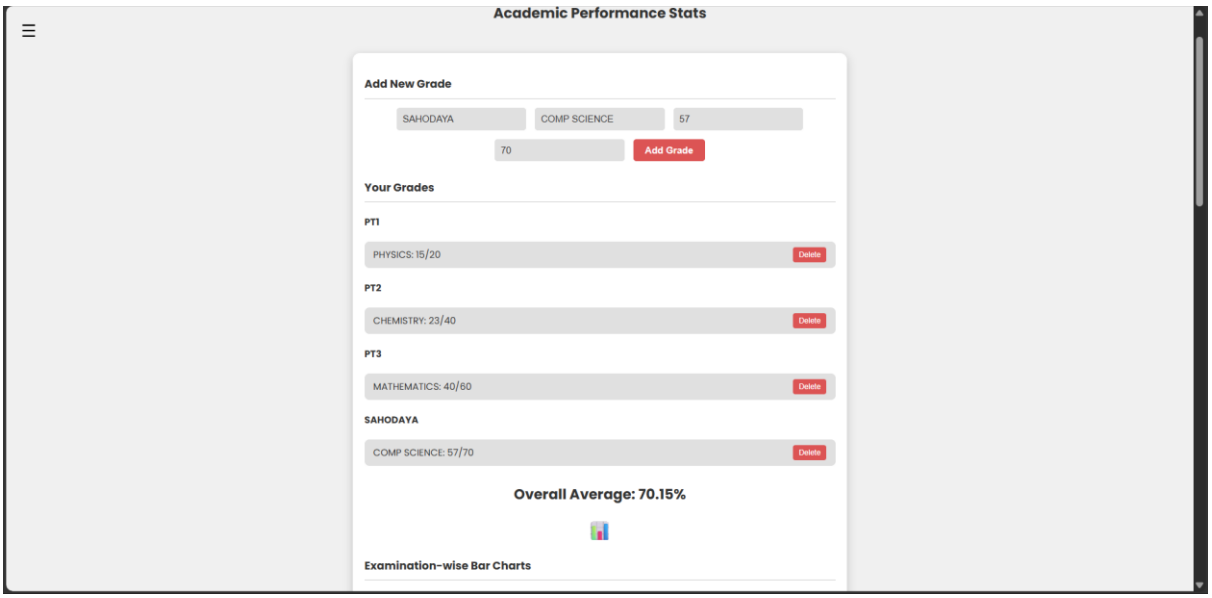
Dashboard



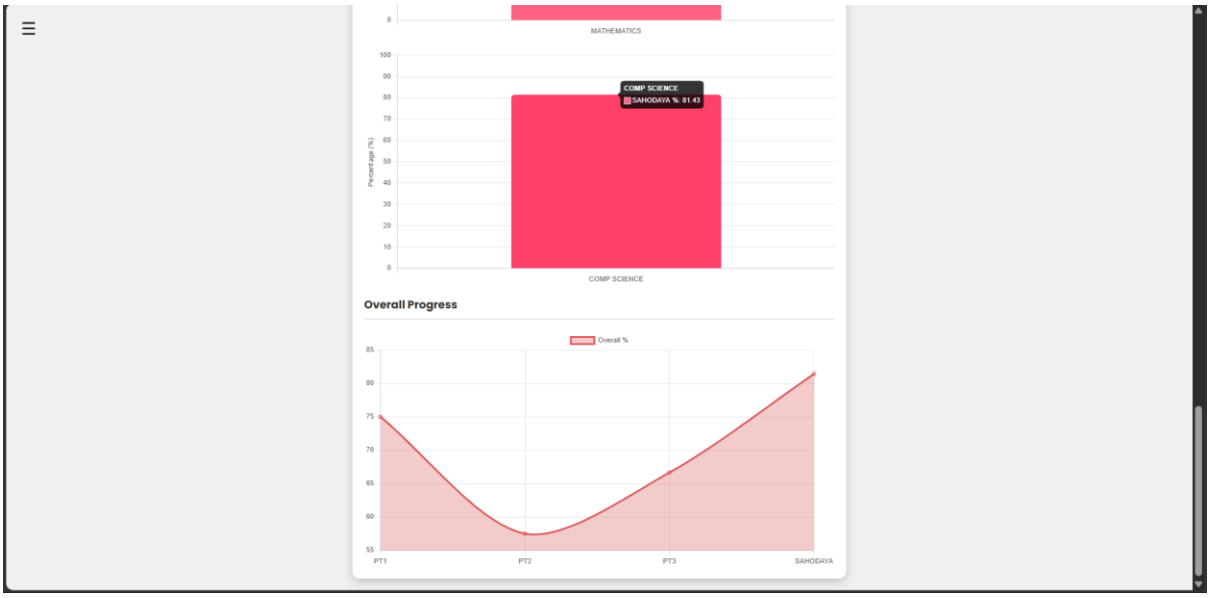
Focus Timer



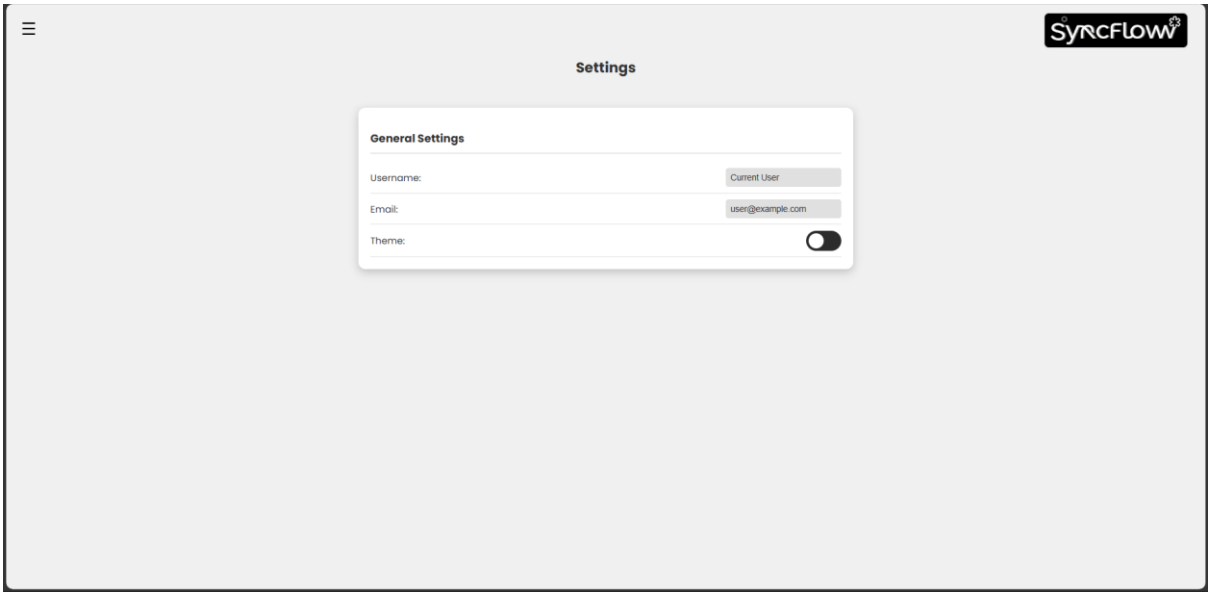
Academic Performance Statistics



Graphical Representation of Perfomance



Settings



FUTURE IMPROVEMENTS

1. **Notes Section**
   a. Add notes for each task.

2. **Subject-Wise Task Sorting**
   a. Separate tasks for each subject.

3. **Record Lectures**
   a. A feature that helps students record their lectures and rewatch them.
4. **Study material and PDFs**
   a. A page that has all study material and PDFs required to make it easier for students to access.
5. **Streak Tracking**
   a. Count how many focus sessions done per day

6. **Compare productivity weekly**

BIBLIOGRAPHY

1. COMPUTER SCIENCE With Python – Sumita Arora

2. Python for Everybody – Charles Severance

WEBLIOGRAPHY

1. https://www.geeksforgeeks.org - Geeks for Geeks
2. https://www.sqlite.org/docs.html - SQL Lite
3. https://www.w3schools.com/python/ - W3Schools Python
4. https://www.gemini.google.com/ - Google Gemini